

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I _____ (Devadharshini E/192524483) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

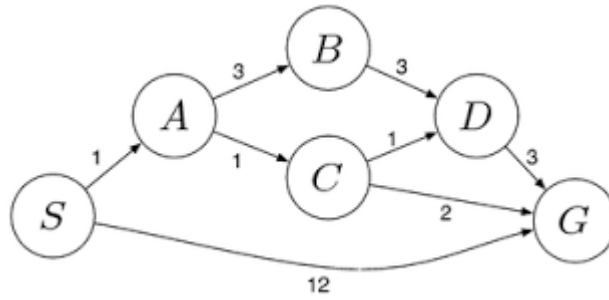
Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

1. Problem Statement

Given: A 4-gallon jug and a 3-gallon jug, neither of which has measuring marks. Water may be filled from a pump, poured from one jug to another, or emptied onto the ground.

Goal: Obtain exactly 2 gallons of water in the 4-gallon jug.

2. Problem Formulation

- **State representation:** Represent a state as (x, y) , where x is the amount of water in the 4-gallon jug and y is the amount in the 3-gallon jug.
- **Initial state:** $(0, 0)$ - both jugs are empty.
- **Goal state:** $(2, 0)$ - exactly 2 gallons are present in the 4-gallon jug.
- **Valid operations:** Fill either jug, empty either jug, pour from the 4-gallon jug to the 3-gallon jug, or pour from the 3-gallon jug to the 4-gallon jug.

3. Step-by-Step Solution

Step	Action	4-Gallon Jug	3-Gallon Jug	State
1	Start with both jugs empty	0	0	$(0, 0)$
2	Fill the 3-gallon jug	0	3	$(0, 3)$
3	Pour the 3-gallon jug into the 4-gallon jug	3	0	$(3, 0)$
4	Fill the 3-gallon jug again	3	3	$(3, 3)$
5	Pour into the 4-gallon jug until it is full	4	2	$(4, 2)$
6	Empty the 4-gallon jug	0	2	$(0, 2)$
7	Pour the remaining 2 gallons into the 4-gallon jug	2	0	$(2, 0)$

4. State-Space Path

$(0, 0) \rightarrow (0, 3) \rightarrow (3, 0) \rightarrow (3, 3) \rightarrow (4, 2) \rightarrow (0, 2) \rightarrow (2, 0)$

5. Pseudocode

```
START
```

```
Jug4 = 0
Jug3 = 0

Fill Jug3
Pour Jug3 into Jug4
Fill Jug3 again
Pour Jug3 into Jug4 until Jug4 is full
Empty Jug4
Pour the remaining water from Jug3 into Jug4

PRINT "Exactly 2 gallons obtained in the 4-gallon jug"

STOP
```

6. Suitable AI Search Method

Breadth-First Search (BFS) is suitable because each amount combination is treated as a state, every valid operation generates a new state, and BFS explores states level by level. Therefore, it can find a solution with the minimum number of operations when all operations have equal cost.

7. Final Result

Final State: (2, 0)

Therefore, the 4-gallon jug contains exactly 2 gallons of water.

8. Conclusion

By systematically filling, pouring and emptying the jugs, the required goal state is reached. The sequence ends at (2, 0), which confirms that exactly 2 gallons are measured in the 4-gallon jug without using any additional measuring device.

Problem 2: Mars Rover Intelligent Agent

A Mars Rover is an autonomous intelligent agent that explores the Martian surface, observes environmental conditions, analyses soil and rock samples, and sends scientific information to Earth.

i. Percepts of the Agent

Percepts are the information received by the rover through its sensors.

Sensor / Source	Percepts Received
Cameras	Images of rocks, soil, terrain, slopes and obstacles
Proximity / LiDAR sensors	Distance from rocks, cliffs and nearby obstacles
Temperature and pressure sensors	Surface temperature and atmospheric pressure
Radiation sensor	Radiation intensity in the operating area
Spectrometer / chemical analyser	Mineral and chemical composition of rocks and soil
Navigation sensors	Position, direction, speed and distance travelled
Gyroscope and inclinometer	Orientation, tilt and slope of the rover
Wheel encoders	Wheel rotation, movement and wheel slippage
Battery and solar sensors	Remaining energy and solar charging status
Communication system	Commands from Earth and transmission status

ii. Characterisation of the Operating Environment

Property	Classification	Reason
Observability	Partially observable	The rover cannot see the complete Martian environment at one time, and some sensor readings may be uncertain.
Determinism	Stochastic	Wheel slip, loose soil, dust and weather can make action outcomes uncertain.

Property	Classification	Reason
Episode type	Sequential	A current movement or sample-selection decision affects later decisions and available energy.
Change	Dynamic	Light, dust, temperature and terrain conditions may change while the rover operates.
State representation	Continuous	Position, distance, temperature, speed and battery level vary continuously.
Agents	Primarily single-agent	The rover completes its mission independently, although it receives high-level instructions from Earth.
Knowledge	Partially known	Some terrain and scientific conditions are unknown until the rover observes them.

iii. Actions the Agent Can Take

- Move forward or backward and turn left or right.
- Stop, slow down, avoid an obstacle or select an alternative route.
- Capture images and examine a scientific target.
- Extend the robotic arm, drill a rock and collect a soil or rock sample.
- Analyse a sample using onboard instruments and store the results.
- Transmit images and scientific data to Earth.
- Recharge using solar energy or enter a low-power/safe mode.
- Update its internal map and plan the next movement.

iv. Performance Evaluation

Performance Measure	Evaluation
Scientific value	Quality and usefulness of samples, images and measurements collected
Safety	Avoidance of collisions, steep slopes, wheel damage and hazardous areas
Energy efficiency	Amount of useful work completed per unit of battery energy
Navigation accuracy	Ability to reach selected targets and follow a safe route

Performance Measure	Evaluation
Time efficiency	Completion of mission activities within the planned time
Communication success	Successful transfer of data to Earth
Reliability and survival	Ability to continue functioning without major failure

$$\text{Performance} = \text{Scientific Value} + \text{Safety} + \text{Accuracy} - \text{Energy Use} - \text{Risk}$$

v. Most Suitable Agent Architecture

The most suitable architecture is a model-based, goal-based, utility-based learning agent.

- A model-based component maintains an internal map and remembers previously observed obstacles and terrain.
- A goal-based component plans actions to reach scientific targets and complete assigned tasks.
- A utility-based component compares routes and actions using safety, energy, time, distance and scientific value.
- A learning component improves navigation and decision-making using previous experience.

Therefore, the rover uses a hybrid architecture because a simple reflex agent cannot safely operate in a partially observable and uncertain environment.

Problem 3: Eight-Queens Problem

The objective is to place eight queens on an 8 x 8 chessboard so that no two queens attack each other. Two queens attack each other when they occupy the same row, the same column or the same diagonal.

Problem Formulation

Component	Formulation
Initial state	An empty 8 x 8 chessboard with no queens placed.
State	A partial arrangement containing one queen in each of the first k columns.
State representation	$Q = [r_1, r_2, \dots, r_k]$, where r_i is the row occupied by the queen in column i .
Actions	Place a queen in the next column in any row that is not attacked by an already placed queen.
Transition model	Add the selected safe row to the current arrangement.
Goal test	Eight queens are placed and no pair shares a row, column or diagonal.
Path cost	Each queen placement has unit cost; a complete arrangement has cost 8.

Conflict Conditions

For queens at positions (r_i, i) and (r_j, j) , the arrangement is valid only when:

$$r_i \neq r_j \text{ and } |r_i - r_j| \neq |i - j|$$

The first condition prevents a row conflict, and the second prevents a diagonal conflict. A separate column conflict does not occur because exactly one queen is placed in each column.

Search Space and Suitable Strategy

A naive formulation allows 8 choices in each of 8 columns, producing $8^8 = 16,777,216$ arrangements. Restricting the board to one queen per row and column gives $8! = 40,320$ permutations. Backtracking reduces the search further by rejecting a partial arrangement immediately after a conflict appears.

Depth-First Search with backtracking is the most suitable strategy because it uses little memory, places one queen at a time and reverses a choice as soon as no safe continuation exists.

One Valid Solution

Q = [1, 5, 8, 6, 3, 7, 2, 4]

	1	2	3	4	5	6	7	8
1	Q							
2							Q	
3					Q			
4								Q
5		Q						
6				Q				
7						Q		
8			Q					

Backtracking Pseudocode

```

FUNCTION SolveQueens(column)
  IF column > 8 THEN
    PRINT board
    RETURN TRUE
  END IF

  FOR row = 1 TO 8
    IF IsSafe (row, column) THEN
      Place queen at (row, column)
      IF SolveQueens (column + 1) = TRUE THEN
        RETURN TRUE
      END IF
      Remove queen from (row, column) // backtrack
    END IF
  END FOR
  RETURN FALSE
END FUNCTION

FUNCTION IsSafe (row, column)
  FOR previousColumn = 1 TO column - 1
    previousRow = position[previousColumn]
    IF previousRow = row THEN RETURN FALSE
    IF ABS (previousRow - row) = ABS(previousColumn - column)
  THEN

```

```
        RETURN FALSE
    END IF
END FOR
RETURN TRUE
END FUNCTION
```

Result: All eight queens are placed without row, column or diagonal conflicts.

Problem 4: OLA Cab Booking Problem-Solving Agent

A customer enters a pickup point and destination and chooses among cab types such as Micro, Mini, Sedan, Shared and Prime. The agent must help the customer reach the destination while considering fare, waiting time, comfort, capacity and route conditions.

Type of Agent

The suitable agent is a goal-based problem-solving agent supported by utility-based selection.

- The goal-based component plans how to move the customer from the pickup point to the destination.
- The utility-based component ranks available cabs by fare, waiting time, comfort, capacity, driver rating and customer preference.
- The route search component chooses a suitable path based on distance, traffic and travel time.

Problem Formulation

Component	OLA Formulation
Initial state	Customer is at the pickup location; destination is entered; no cab has been booked.
Goal state	Customer reaches the destination safely within acceptable cost and time.
State space	Pickup/destination locations, available cabs, possible routes, traffic levels, fares and waiting times.
Actions	Enter locations, view cabs, select cab, select route, confirm booking, travel and pay.
Transition model	Each action changes the booking or travel state, such as unbooked -> booked -> picked up -> destination reached.
Path cost	A combination of fare, waiting time and travel time.
Performance measure	Low cost and waiting time, high safety, comfort, availability and customer satisfaction.

Utility Function

$$\text{Utility} = w1(\text{Comfort}) + w2(\text{Safety}) + w3(\text{Rating}) - w4(\text{Fare}) - w5(\text{Waiting Time})$$

The weights w_1 to w_5 represent the importance assigned by the customer. For example, a budget traveller gives a larger weight to fare, whereas a family traveller may give larger weights to comfort and capacity.

Cab Recommendation Examples

Customer Requirement	Possible Recommendation
Lowest fare	Shared or Micro
Family with luggage	Sedan or SUV
High comfort	Prime
Single passenger	Mini, Auto or Bike
Urgent pickup	Nearest available cab
More passengers	Cab with sufficient seating capacity

Pseudocode

```
START
INPUT pickup_location, destination
INPUT passenger_count, maximum_budget
INPUT comfort_preference, luggage_requirement

available_cabs = GET_AVAILABLE_CABS(pickup_location)
IF available_cabs is empty THEN
    PRINT "No cabs available"
    STOP
END IF

best_cab = NULL
best_utility = -INFINITY

FOR each cab in available_cabs
    fare = CALCULATE_FARE(pickup_location, destination, cab.type)
    waiting_time = cab.arrival_time
    comfort = cab.comfort_level
    rating = cab.driver_rating

    IF cab.capacity >= passenger_count AND fare <= maximum_budget
    THEN
        utility = comfort_preference * comfort + rating - fare - waiting_time
        IF utility > best_utility THEN
            best_utility = utility
            best_cab = cab
        END IF
    END IF
END FOR

IF best_cab is not NULL THEN
    DISPLAY best_cab, fare and waiting_time
    IF customer confirms THEN BOOK best_cab
ELSE
    PRINT "No cab matches the requirements"
END IF
STOP
```

Result: The agent recommends and books the most suitable available cab while working toward the goal of reaching the destination.

Problem 5: Uniform Cost Search for the Warehouse Network

The logistics company must find the least-cost path from warehouse S to destination G. Each edge cost represents the combined fuel and time cost. Uniform Cost Search (UCS) always expands the frontier node with the smallest cumulative path cost.

Weighted Routes

Route	Cost
S -> A	1
S -> G	12
A -> B	3
A -> C	1
B -> D	3
C -> D	1
C -> G	2
D -> G	3

Step-by-Step UCS Execution

Step	Expanded Node	Cumulative Cost	Frontier after Expansion
1	S	0	A:1, G:12
2	A	1	C:2, B:4, G:12
3	C	2	D:3, B:4, G:4
4	D	3	B:4, G:4 (new D->G cost 6 is rejected)
5	B	4	G:4 (new B->D cost 7 is rejected)
6	G	4	Goal reached

Path-Cost Comparison

Possible Path	Total Cost
S -> G	12

Possible Path	Total Cost
S -> A -> B -> D -> G	$1 + 3 + 3 + 3 = 10$
S -> A -> C -> D -> G	$1 + 1 + 1 + 3 = 6$
S -> A -> C -> G	$1 + 1 + 2 = 4$

Optimal Result

Least-Cost Path: S -> A -> C -> G

Minimum Cost = 4

UCS terminates when G is removed from the priority queue with cost 4. Because all edge costs are non-negative, this is guaranteed to be the least-cost path.

Uniform Cost Search Pseudocode

```

FUNCTION UniformCostSearch(start, goal)
    frontier = priority queue ordered by path cost
    INSERT (start, cost = 0, path = [start]) into frontier
    best_cost[start] = 0

    WHILE frontier is not empty
        current = REMOVE_LOWEST_COST(frontier)
        node = current.node

        IF node = goal THEN
            RETURN current.path, current.cost
        END IF

        FOR each neighbour of node
            new_cost = current.cost + edge_cost(node, neighbour)
            IF neighbour not in best_cost OR new_cost <
best_cost[neighbour] THEN
                best_cost[neighbour] = new_cost
                INSERT (neighbour, new_cost, current.path + neighbour)
            END IF
        END FOR
    END WHILE
    RETURN FAILURE
END FUNCTION

```


Overall Conclusion

The five problems illustrate important foundations of Artificial Intelligence: state-space representation, intelligent agents, backtracking, goal and utility-based reasoning, and cost-based graph search.

Problem	AI Technique	Final Outcome
Water Jug	State-space search / BFS	2 gallons in the 4-gallon jug
Mars Rover	Model-based utility learning agent	Safe and scientifically useful exploration
Eight Queens	DFS with backtracking	Eight non-attacking queens
OLA Booking	Goal-based and utility-based agent	Best suitable cab and route
Warehouse Routing	Uniform Cost Search	S -> A -> C -> G, cost 4